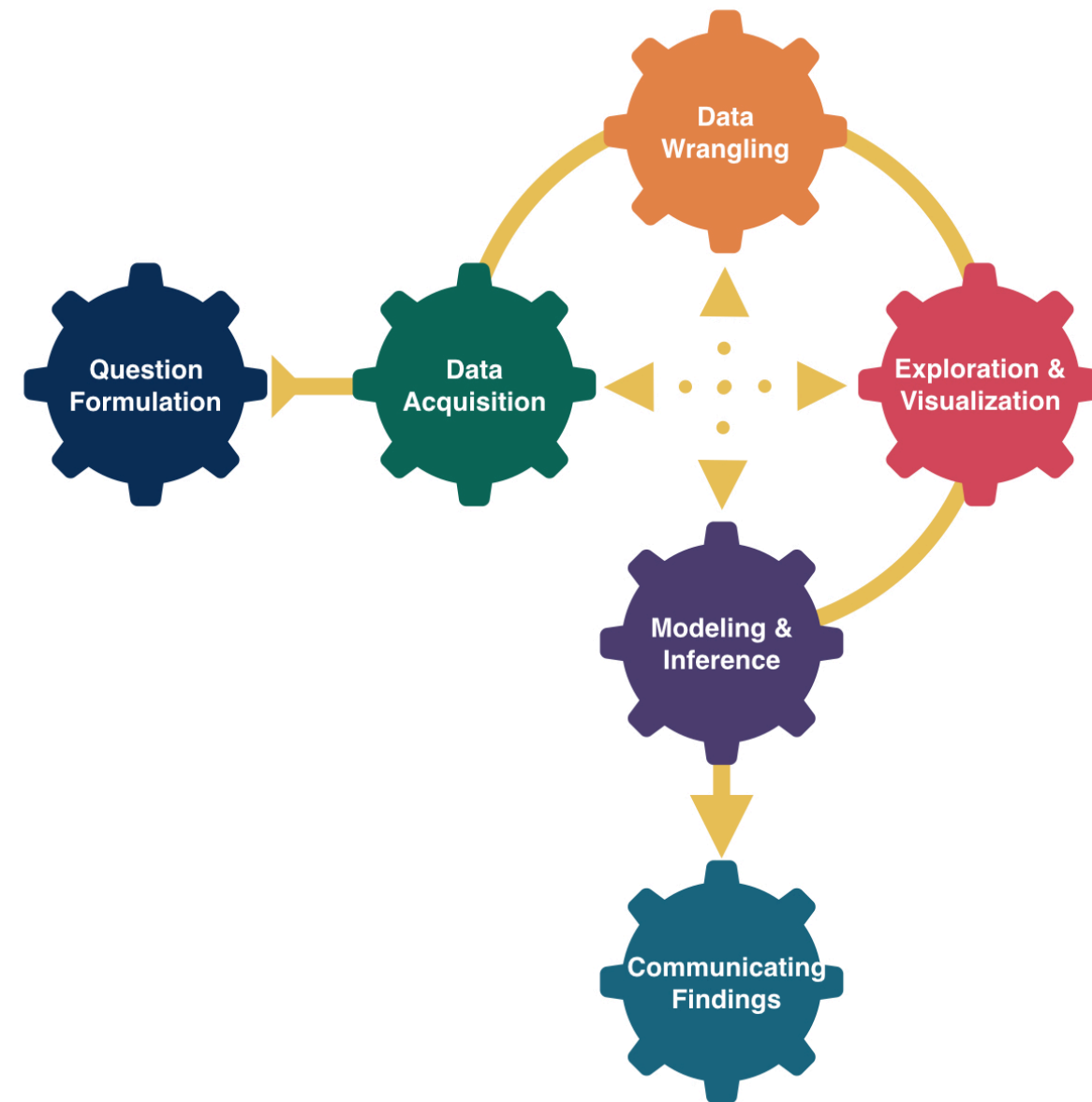


Quantifying Uncertainty



Kelly McConville

DATA 100

Week 8 | Spring 2026

Announcements

- Remember that the code in your problem sets should be your own.
- Load the relevant packages at the top of the Quarto document.

```
1 library(tidyverse)
2 library(moderndiver)
```

Goals for Today

- Models beyond linear regression
- Modeling & Ethics: Algorithmic bias
- Quantifying uncertainty
- Sampling variability
- Sampling distributions

Beyond Linear Regression

Neural Networks

- What people are talking about right now when they say “*AI models*”.
- Linear regression on steroids:
 - Layers and layers of transformed linear regression models.
 - In simple linear regression we have two model parameters: β_0 and β_1 .
 - In Gemini 3 Flash, there are between 1.2 and 1.8 trillion parameters.
- Won't fit in DATA 100 but will interact with fitted models.

Regression Trees

- Useful if
 - Have a lot of categorical variables (with many categories).
 - Expect local interaction effects.
- When the response variable is categorical, then called **Classification Trees**.
- Let's explore fitting and interpreting these models.

Example: Return to the Candy Example

- Want to predict when a candy will win!
- Let's focus on just the categorical explanatory variables.
 - Though trees can handle both types.

```
1 # Wrangling the data
2 candy <- read_csv("https://raw.githubusercontent.com/fivethirtyeight/data/master/candy-power-ranking/candy-data.csv")
3 mutate(pricepercent = pricepercent*100) %>%
4 select(-competitorname, - sugarpercent, -pricepercent) %>%
5 mutate(across(.cols = -winpercent, .fns = ~
6           case_when(.x == 1 ~ "yes",
7                     .x == 0 ~ "no")))
8
9 glimpse(candy)
```

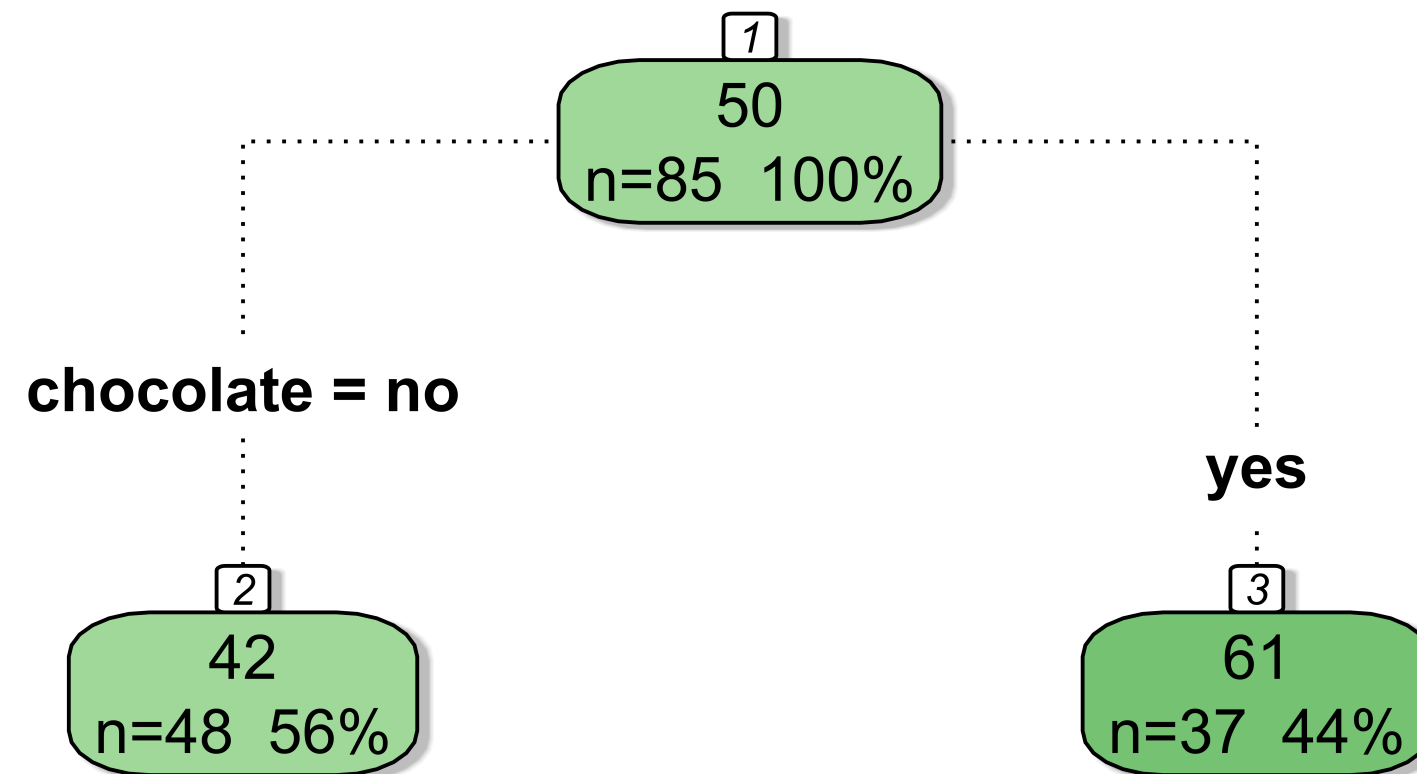
Rows: 85

Columns: 10

\$ chocolate	<chr>	"yes", "yes", "no", "no", "no", "yes", "yes", "no", "no", "no"
\$ fruity	<chr>	"no", "no", "no", "no", "yes", "no", "no", "no", "no", "no"
\$ caramel	<chr>	"yes", "no", "no", "no", "no", "no", "yes", "no", "no", "no"
\$ peanutyalmondy	<chr>	"no", "no", "no", "no", "no", "yes", "yes", "yes", "no", "no"
\$ nougat	<chr>	"no", "yes", "no", "no", "no", "no", "yes", "no", "no", "no"
\$ crispedricewafer	<chr>	"yes", "no", "no", "no", "no", "no", "no", "no", "no", "no"
\$ hard	<chr>	"no", "no", "no", "no", "no", "no", "no", "no", "no", "no"
\$ bar	<chr>	"yes", "yes", "no", "no", "no", "yes", "yes", "no", "no", "no"
\$ pluribus	<chr>	"no", "no", "no", "no", "no", "no", "no", "yes", "yes", "yes"
\$ winpercent	<dbl>	66.97173, 67.60294, 32.26109, 46.11650, 52.34146, 50.00000, 50.00000, 50.00000, 50.00000, 50.00000

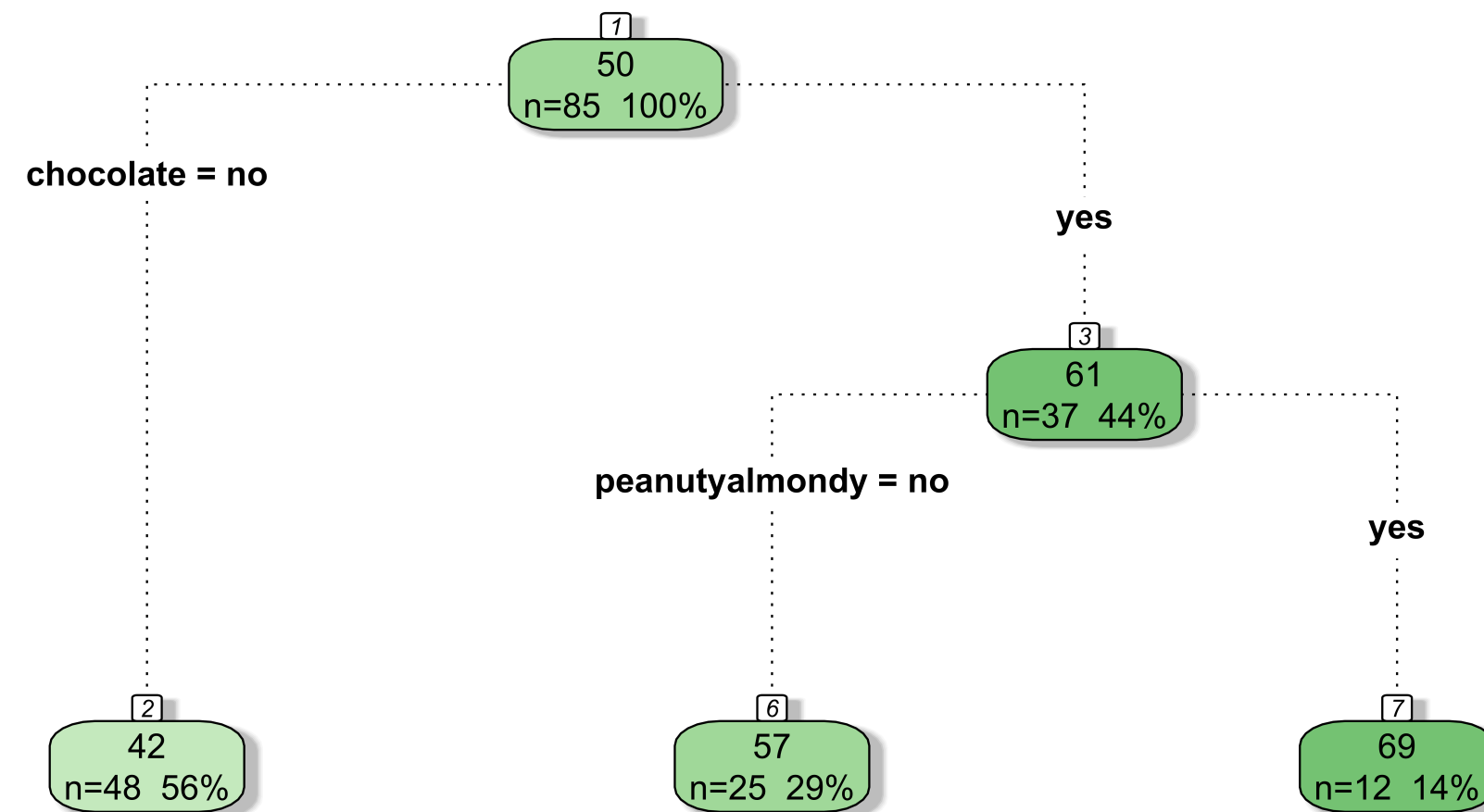
Regression Trees

- Model form is very different from linear regression.
- Split sample into two groups based on an explanatory variable.
 - Wants groups to be homogeneous as possible for the response variable.



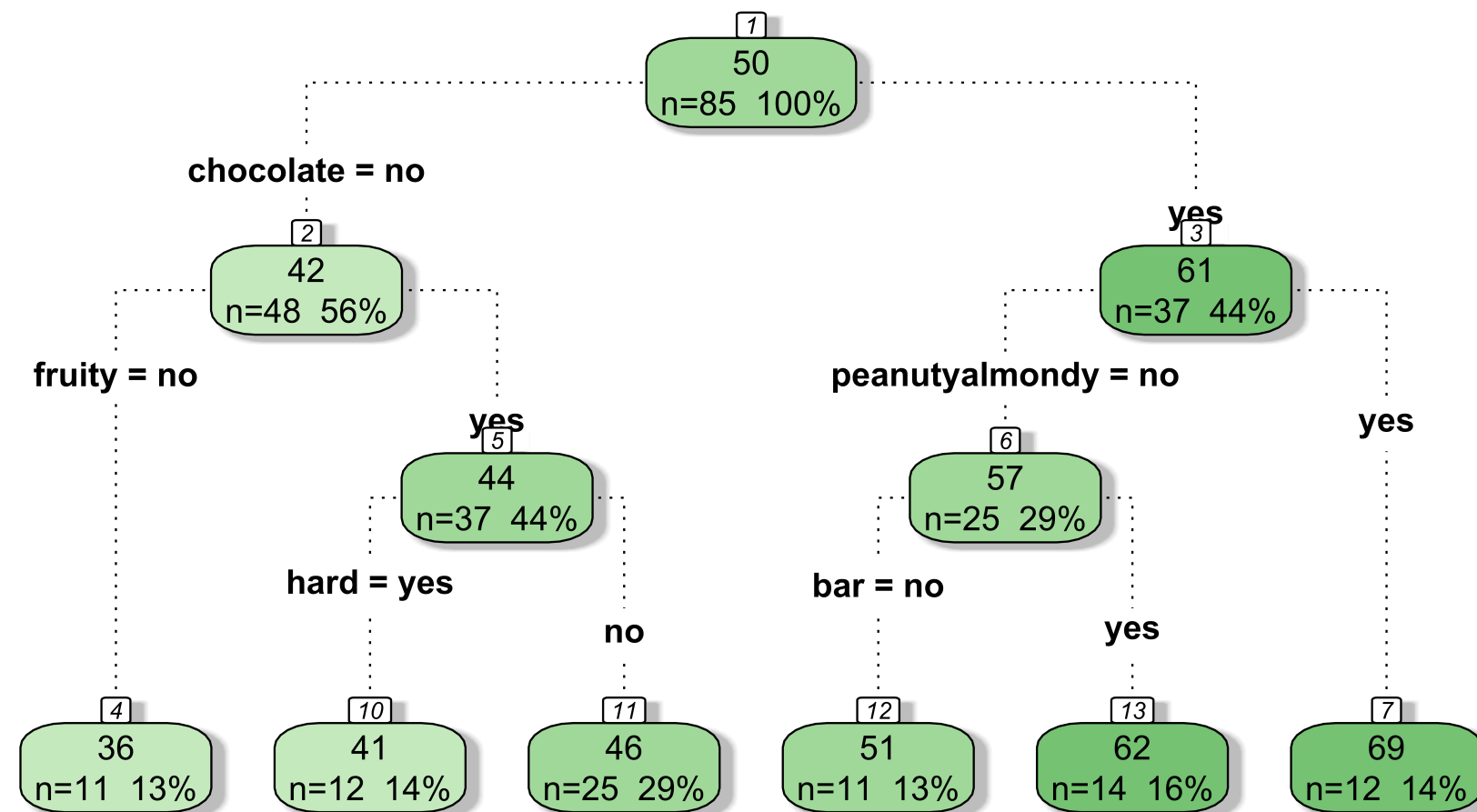
Regression Trees

- Split the subgroups to make subsubgroups that are even more homogeneous.



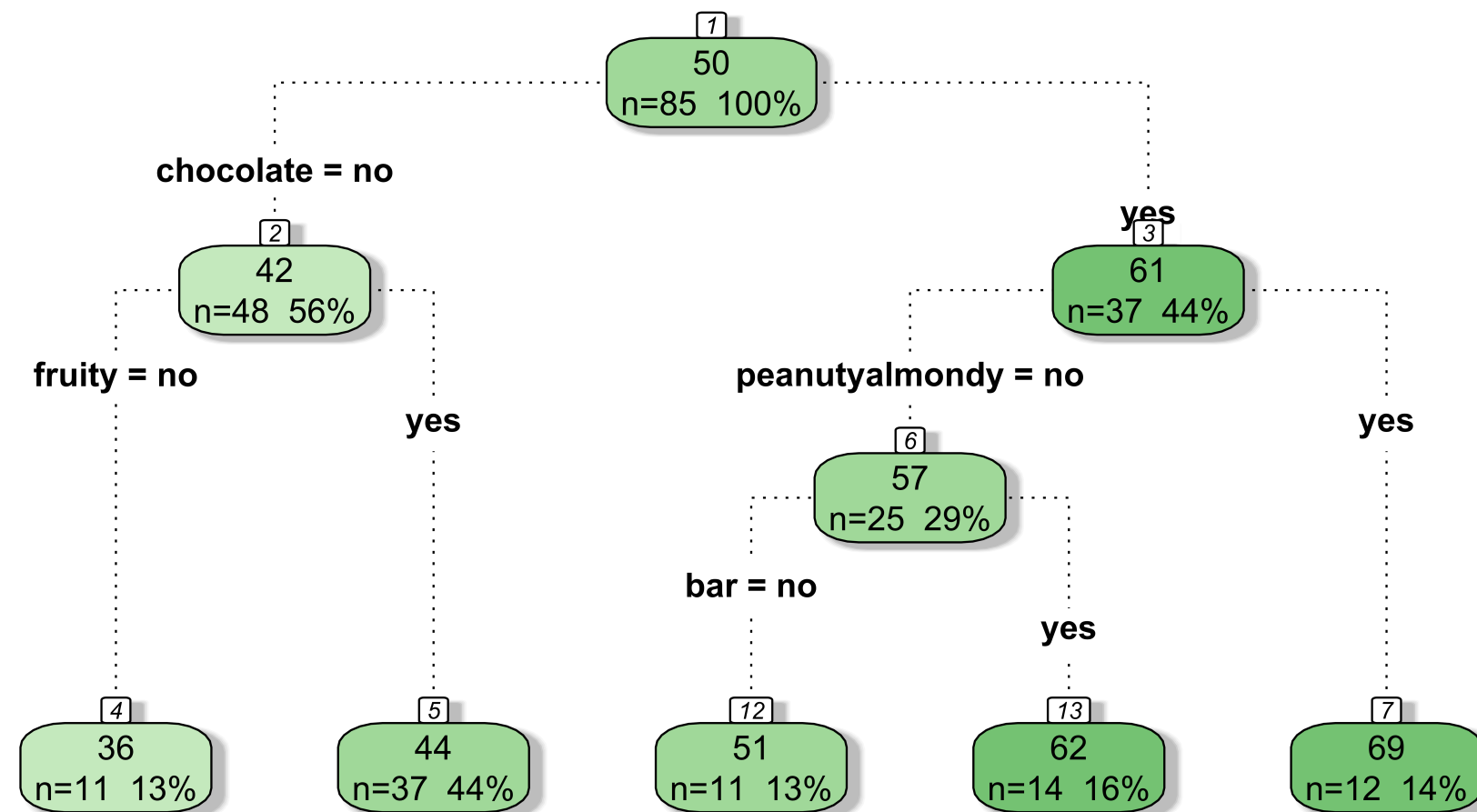
Regression Trees

- Keep going until you have a big tree.



Regression Trees

- Then prune back the splits that aren't very predictive/useful to get the optimal tree.



Coding Regression Trees

Let's see how to code up what we just did.

```
1 # Package to fit the tree
2 library(rpart)
3
4 # Package to visualize the tree
5 library(rattle)
6
7 # Set the random seed so that I get the same tree each time
8 set.seed(3344)
```

Coding Regression Trees

- Fit the regression tree.
- **cp** = complexity parameter
 - **cp = 0** (Grow a very large tree).
 - **minsplit** = smallest tree allowed.
 - **minbucket** = how many cases need to be in each bucket of the tree.

```
1 candy_tree <- rpart(winpercent ~ ., data = candy,  
2                     method = "anova",  
3                     control = rpart.control(cp = 0,  
4                                             minsplit = 2,  
5                                             minbucket = 10))
```

Coding Regression Trees

- Look at the amount of error for different cost penalties.
- Pick the **cp** where the **xerror** is the lowest (or nearly the lowest).

```
1 printcp(candy_tree)
```

Regression tree:

```
rpart(formula = winpercent ~ ., data = candy, method = "anova",  
      control = rpart.control(cp = 0, minsplit = 2, minbucket = 10))
```

Variables actually used in tree construction:

```
[1] bar          chocolate    fruity        hard          peanutyalmondy
```

Root node error: 18187/85 = 213.97

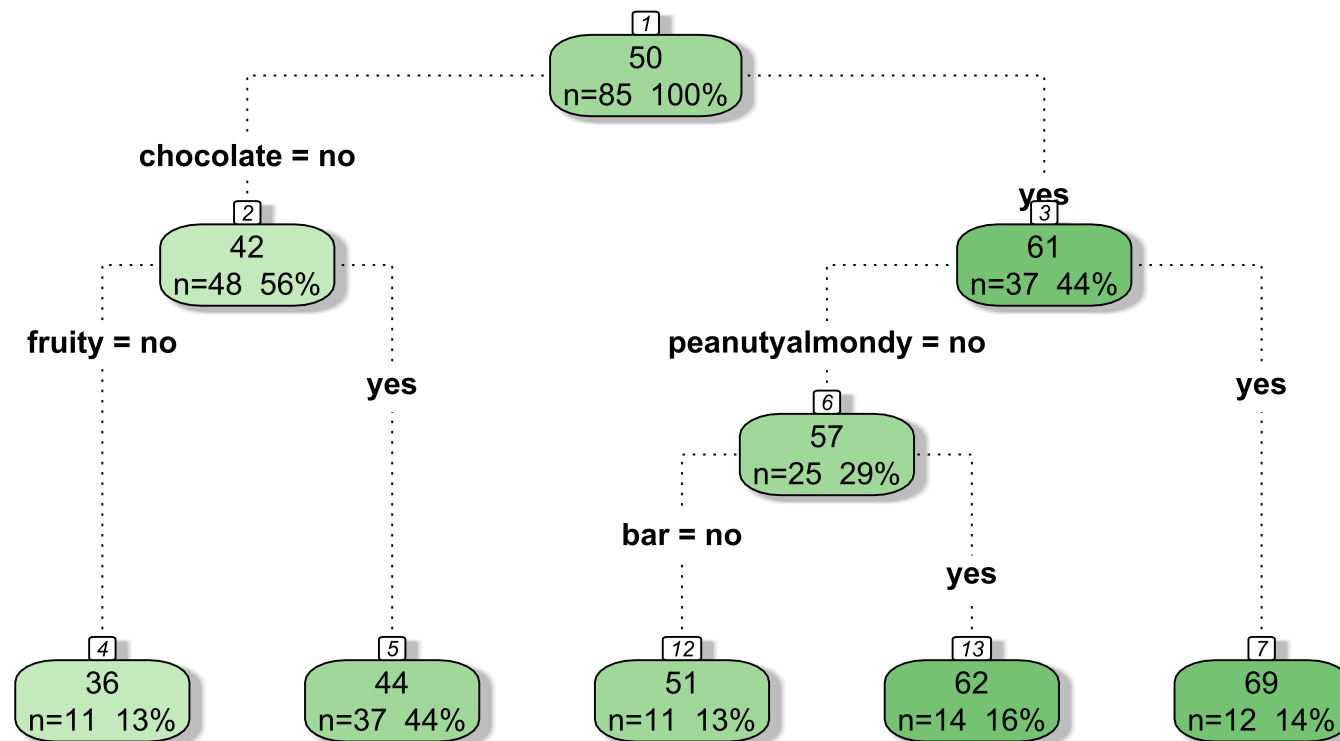
n= 85

	CP	nsplit	rel error	xerror	xstd
1	0.405154	0	1.00000	1.02387	0.130063
2	0.056141	1	0.59485	0.61273	0.083460
3	0.044007	2	0.50071	0.60700	0.080000

Coding Regression Trees

Best **cp** for this example is just above 0.011297.

```
1 candy_tree_pruned <- prune(candy_tree, cp = 0.011297 +  
2 fancyRpartPlot(candy_tree_pruned, cex = 1, caption =
```



Prediction

Can still use the `predict()` function to predict new observations.

```
1 twix <- data.frame(chocolate = "yes", peanutyalmondy = "no", crispedricewafer = "yes",  
2                     bar = "yes", fruity = "no", caramel = "yes", nougat = "no",  
3                     hard = "no", pluribus = "no")  
4 twix
```

```
  chocolate peanutyalmondy crispedricewafer bar fruity caramel nougat hard  
1        yes              no             yes yes      no      yes    no   no  
  pluribus  
1        no
```

```
1 predict(candy_tree_pruned, newdata = twix)
```

```
1  
62.29856
```

Data Ethics: Algorithmic Bias

Return to the American Statistical Association's
“Ethical Guidelines for Statistical Practice”

Integrity of Data and Methods

“The ethical statistical practitioner seeks to understand and mitigate known or suspected limitations, defects, or biases in the data or methods and communicates potential impacts on the interpretation, conclusions, recommendations, decisions, or other results of statistical practices.”

“For models and algorithms designed to inform or implement decisions repeatedly, develops and/or implements plans to validate assumptions and assess performance over time, as needed. Considers criteria and mitigation plans for model or algorithm failure and retirement.”

Algorithmic Bias

Algorithmic bias: when the model systematically creates unfair outcomes, such as privileging one group over another.

Example: The Coded Gaze



Joy Buolamwini

- Facial recognition software struggles to see faces of color.
- Algorithms built on a non-diverse, biased dataset.

Algorithmic Bias

Algorithmic bias: when the model systematically creates unfair outcomes, such as privileging one group over another.

Example: COMPAS model used throughout the country to predict recidivism

- Differences in predictions across race and gender

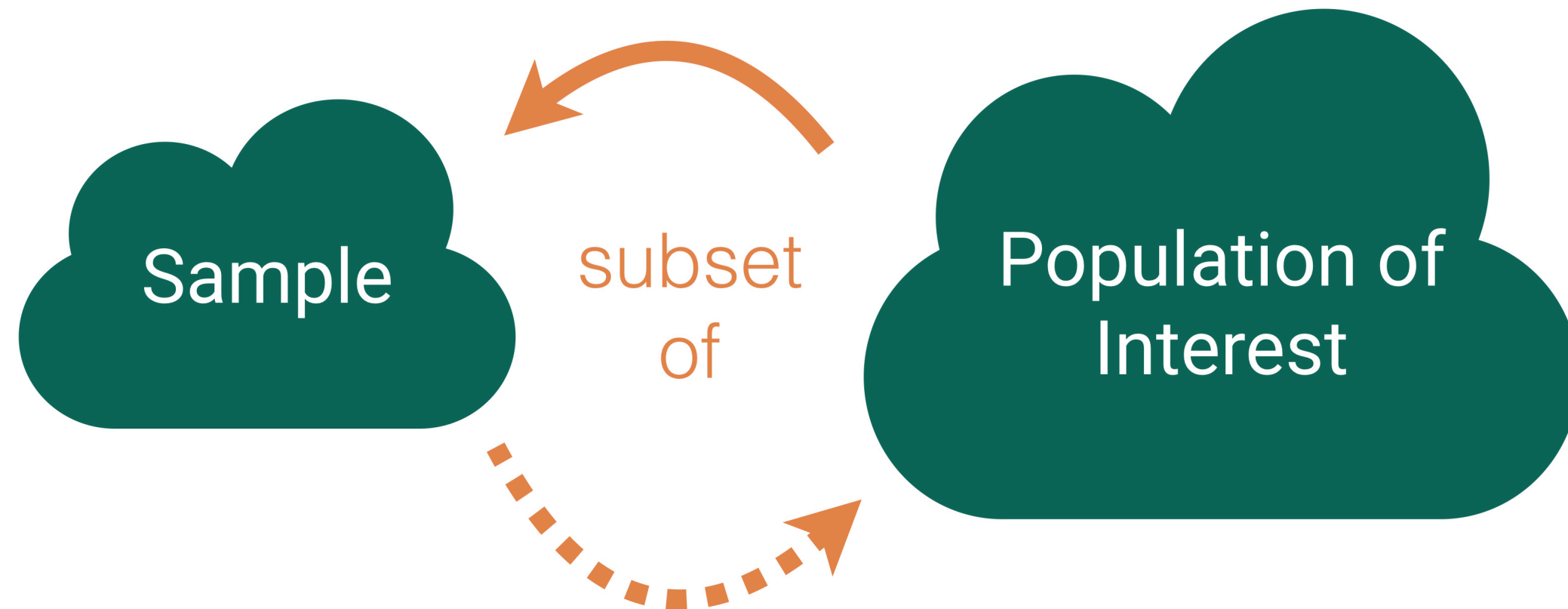
Prediction Fails Differently for Black Defendants		
	WHITE	AFRICAN AMERICAN
Labeled Higher Risk, But Didn't Re-Offend	23.5%	44.9%
Labeled Lower Risk, Yet Did Re-Offend	47.7%	28.0%

Overall, Northpointe's assessment tool correctly predicts recidivism 61 percent of the time. But blacks are almost twice as likely as whites to be labeled a higher risk but not actually re-offend. It makes the opposite mistake among whites: They are much more likely than blacks to be labeled lower risk but go on to commit other crimes. (Source: ProPublica analysis of data from Broward County, Fla.)

ProPublica Analysis

Shift Gears: Statistical Inference

The ❤️ of statistical inference is quantifying uncertainty



```
1 library(tidyverse)
2 ce <- read_csv("data/fmli.csv")
3 summarize(ce, meanFINCBTAX = mean(FINCBTAX))
```

```
# A tibble: 1 × 1
  meanFINCBTAX
    <dbl>
1      62480.
```

The ❤️ of statistical inference is quantifying uncertainty

```
1 library(tidyverse)
2 ce <- read_csv("data/fmli.csv")
3 summarize(ce, meanFINCBTAX = mean(FINCBTAX))
```

```
# A tibble: 1 × 1
  meanFINCBTAX
    <dbl>
1      62480.
```

Like with regression, need to distinguish between the **population** and the **sample**

- **Parameters:**

- Based on the **population**
- Unknown then if don't have data on the whole population
- EX: β_0 and β_1
- EX: μ = population mean

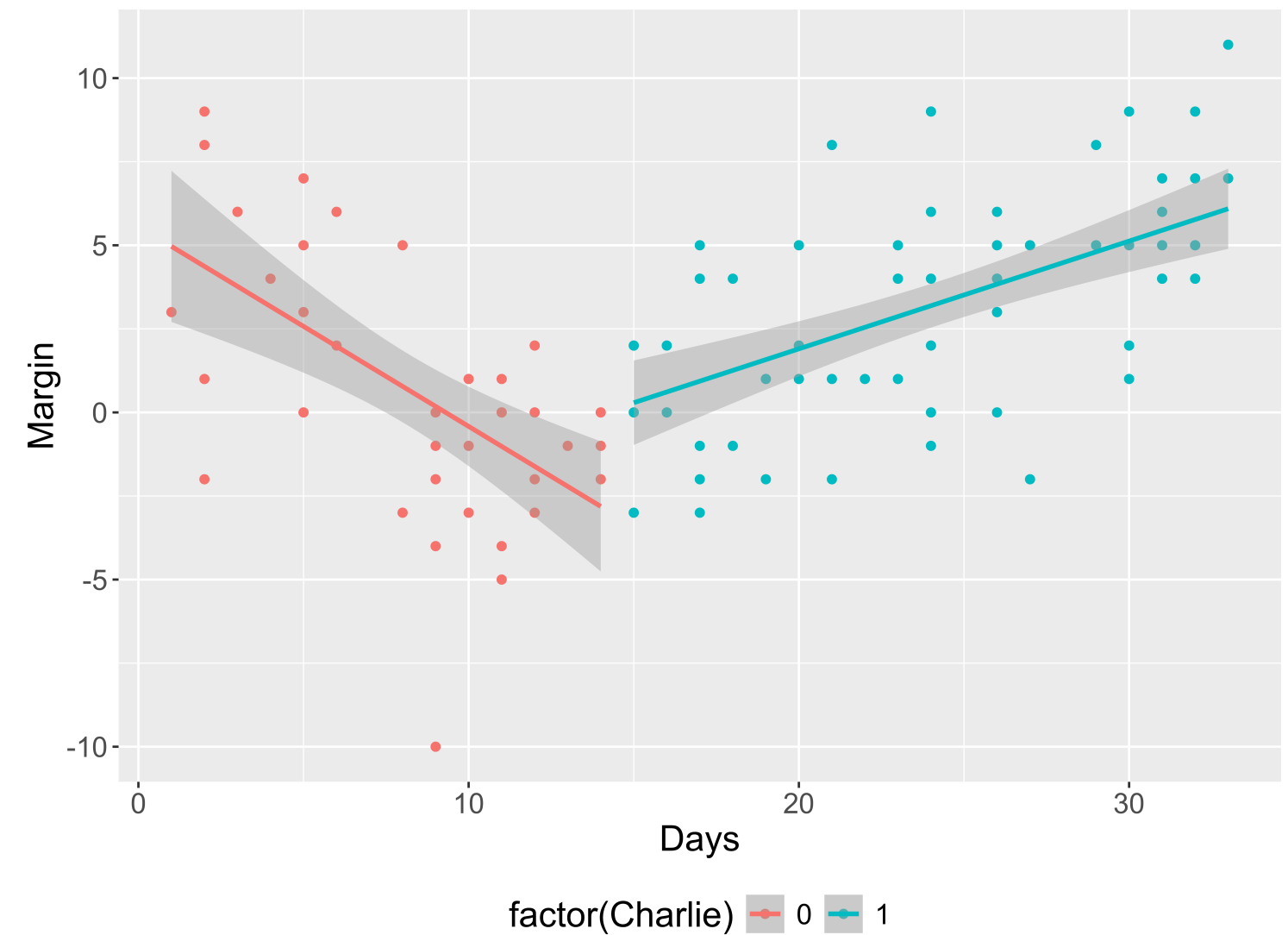
- **Statistics:**

- Based on the **sample** data
- Known
- Usually estimate a population parameter
- EX: $\hat{\beta}_0$ and $\hat{\beta}_1$
- EX: $\bar{x}$ = sample mean

Quantifying Our Uncertainty

R has been giving us uncertainty estimates:

```
1 library(Stat2Data)
2 data("Pollster08")
3
4 ggplot(Pollster08, aes(x = Days,
5                       y = Margin,
6                       color = factor(Charlie))) +
7   geom_point() +
8   stat_smooth(method = "lm", se = TRUE) +
9   theme(legend.position = "bottom")
```



Quantifying Our Uncertainty

R has been giving us uncertainty estimates:

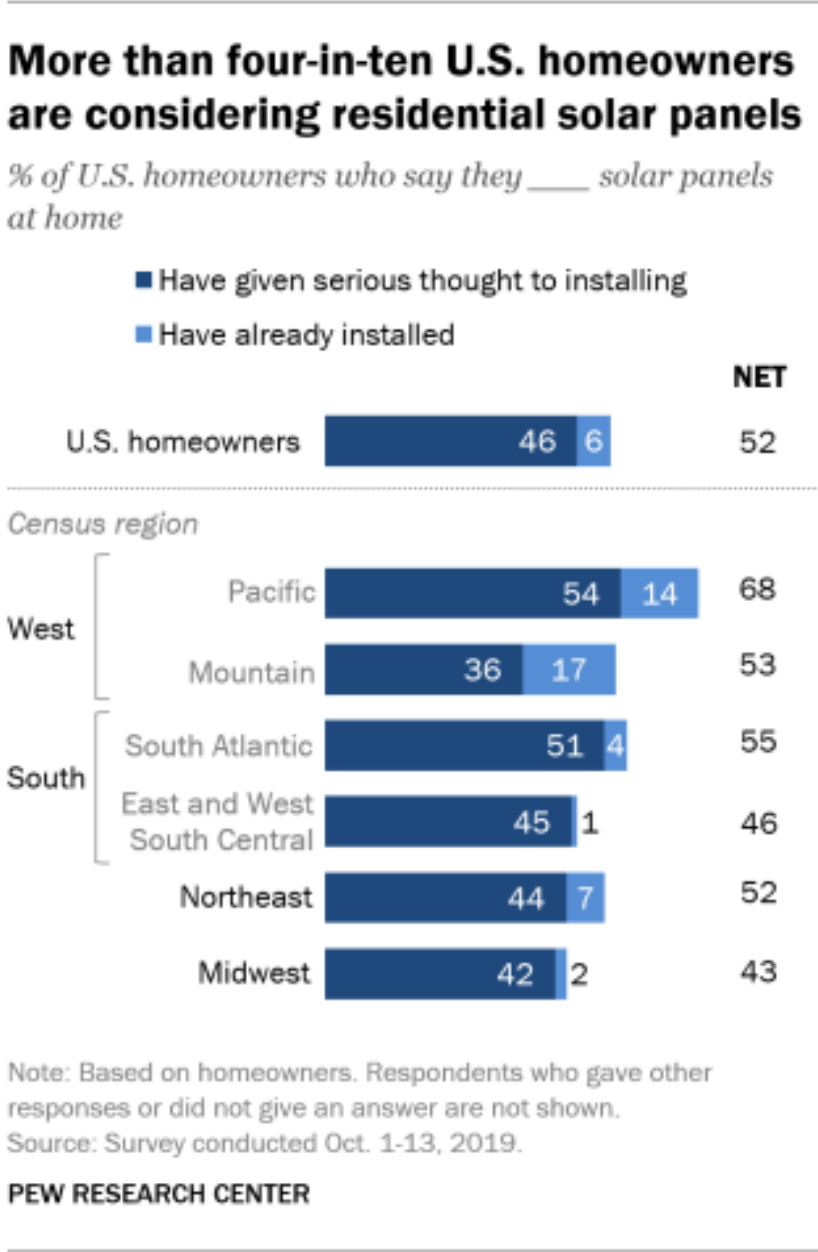
```
1 library(Stat2Data)
2 data("Pollster08")
3 modPoll <- lm(Margin ~ Days*factor(Charlie), data = Pollster08)
4 library(moderndive)
5 get_regression_table(modPoll)
```

A tibble: 4 × 7

	term	estimate	std_error	statistic	p_value	lower_ci	upper_ci
	<chr>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>
1	intercept	5.57	1.09	5.11	0	3.40	7.73
2	Days	-0.598	0.121	-4.96	0	-0.838	-0.359
3	factor(Charlie): 1	-10.1	1.92	-5.25	0	-13.9	-6.29
4	Days:factor(Charlie)1	0.921	0.136	6.75	0	0.65	1.19

Quantifying Our Uncertainty

The news and journal articles are also giving us uncertainty estimates:



Note: The findings are based on a [survey](#) conducted Oct. 1-13, 2019, among 3,627 U.S. adults on Pew Research Center’s American Trends Panel. The margin of sampling error for the full sample is plus or minus 2.1 percentage points. The margin of error for the 2,564 U.S. homeowners is plus or minus 2.5 percentage points. See [full topline results](#).

Quantifying Our Uncertainty

The news and journal articles are also giving us uncertainty estimates:

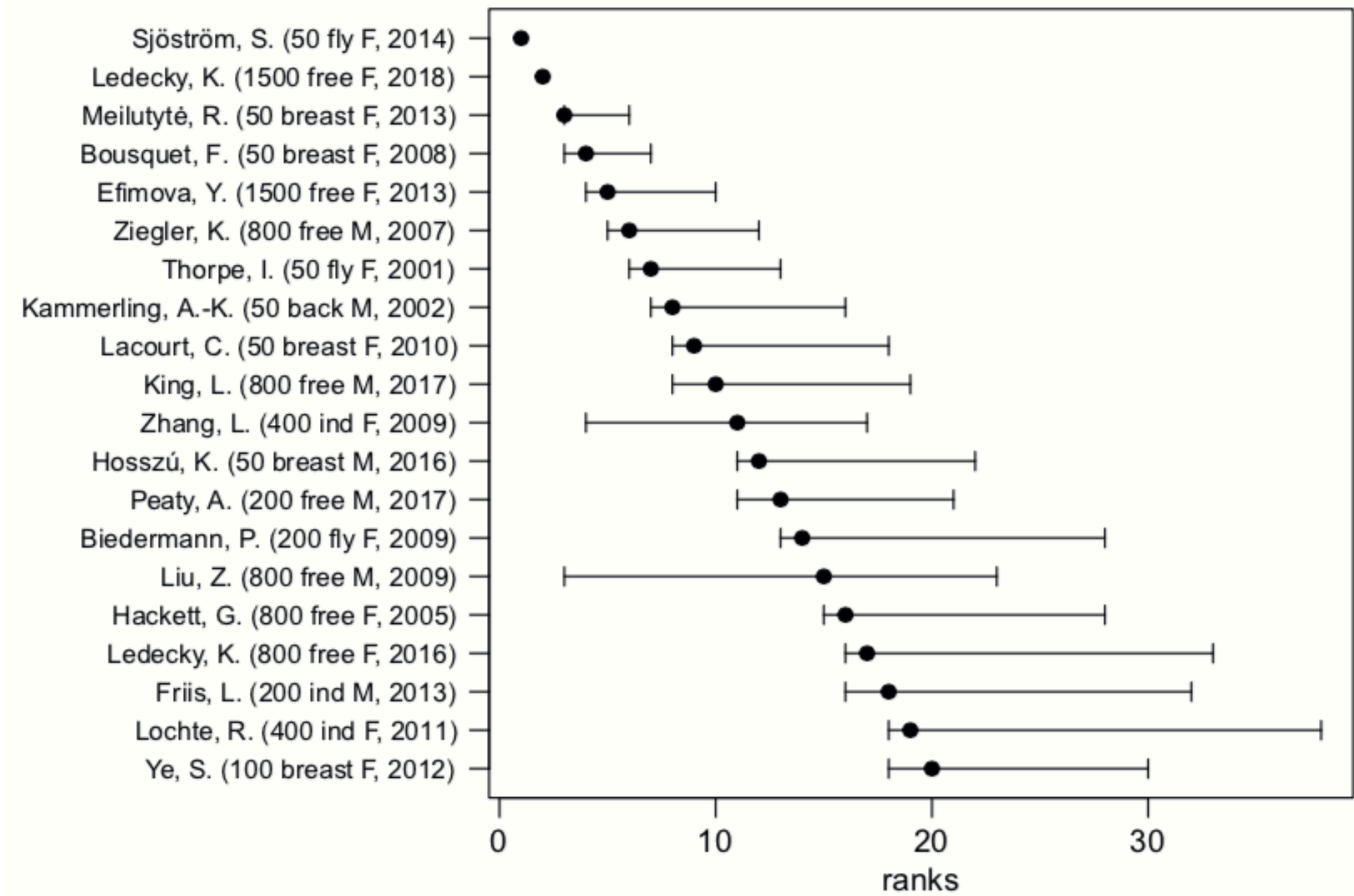


Figure 2: Ranking of the top 20 personal best swims (and their swimmers), 2001–2018, with 95% confidence intervals. Better-ranked swimmers are higher on the y-axis.

Statistical Inference

Goal: Draw conclusions about the population based on the sample.

Main Flavors

- Estimating numerical quantities (parameters).
- Testing conjectures.

Estimation

Goal: Estimate a (population) parameter.

Best guess?

- The corresponding (sample) statistic

Example: What one word would you use to describe Bucknell?

Estimation

Key Question: How accurate is the statistic as an estimate of the parameter?

Helpful Sub-Question: If we take many samples, how much would the statistic vary from sample to sample?

Need two new concepts:

- The **sampling variability** of a statistic
- The **sampling distribution** of a statistic

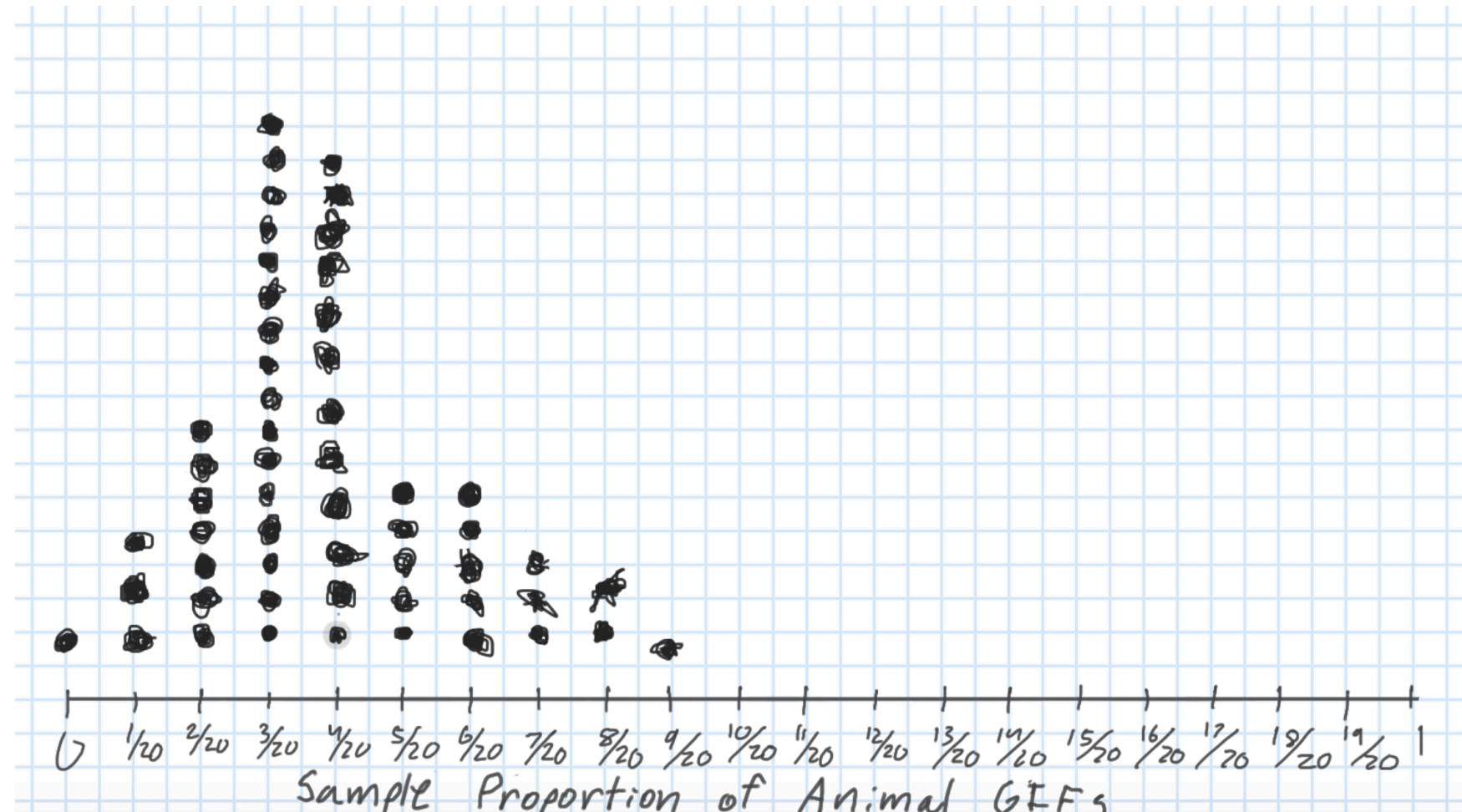
**Let's learn about these ideas
through an activity!**

Sampling Distribution of a Statistic

Steps to Construct an (Approximate) Sampling Distribution:

1. Decide on a sample size, n .
2. Randomly select a sample of size n from the population.
3. Compute the sample statistic.
4. Put the sample back in.
5. Repeat Steps 2 - 4 many (1000+) times.

Sampling Distribution of a Statistic



- Center? Shape?
- Spread?
 - Standard error = standard deviation of the statistic
- What happens to the center/spread/shape as we increase the sample size?
- What happens to the center/spread/shape if the true parameter changes?

